

5954025 - Sistemas Distribuídos

Aula 11a - Exclusão Mútua

Prof. Dr. Denis M. L. Martins

DCM | FFCLRP | USP

Objetivos de Aprendizagem

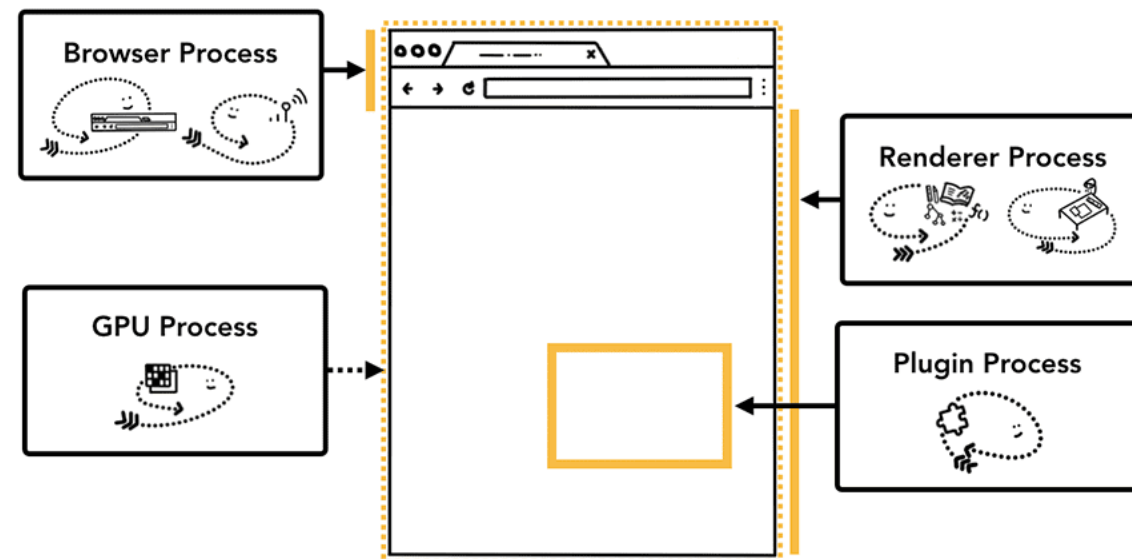
- Entender por que múltiplos processos rodando simultaneamente causam desafios de sincronização.
- Diferenciar concorrência em um único computador e em sistemas distribuídos.
- Compreender a ideia de seção crítica e exclusão mútua.
- Analisar algoritmos descentralizados: votação e token.

Relembrando: Multitarefa

- Os sistemas operacionais permitem que múltiplas computações ocorram **concorrentemente** em um único sistema computacional.
- Processo como unidade de **gerenciamento** e **proteção**.

Na imagem: Processos apontando para diferentes partes da interface do usuário (UI) do navegador.

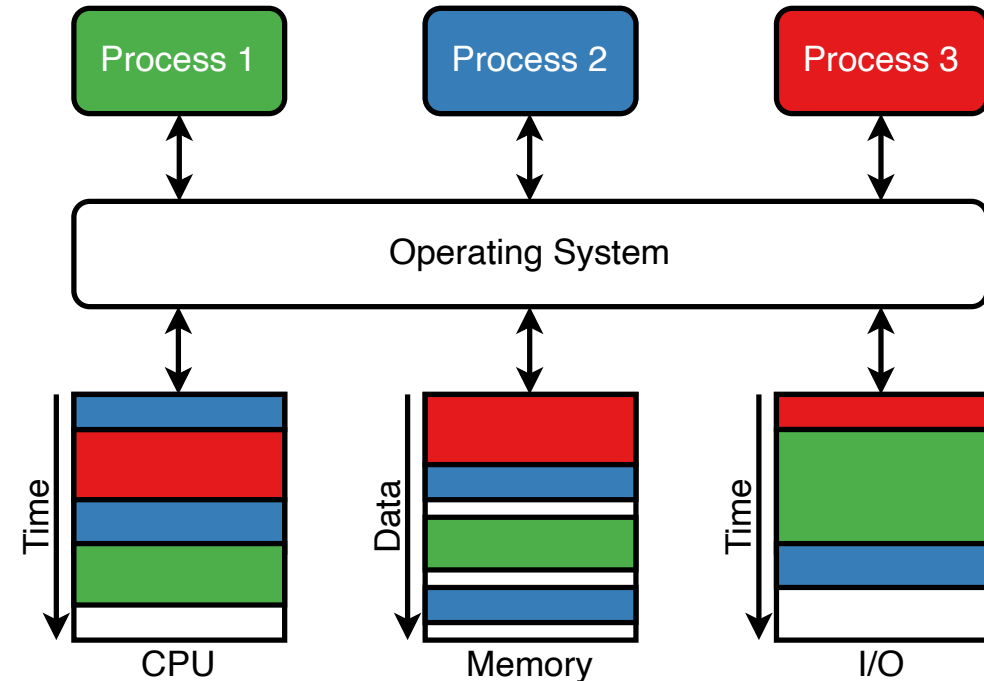
Fonte: [Google Developers](#) 



Relembrando: Concorrência

- Recurso Compartilhado: Dados ou equipamentos acessados por vários processos.
- Mesmo em um único CPU core: ilusão de simultaneidade.
- Vários processos executam **simultaneamente** em um sistema.

Na Imagem: SO gerenciando memória, CPU e I/O para três processos diferentes. Fonte: [OER OS](#) ↗



Relembrando: Condição de Corrida (Race Conditions)

Se não houver um mecanismo de controle ao acesso à variável `next_available_pid`, processos-filhos Processo A e Processo B podem ter o **mesmo** `pid`.

Tempo	Processo A		Processo B
T_1	<code>pid_t child = fork();</code>		<code>pid_t child = fork();</code>
T_2	<code>request pid</code>		<code>request pid</code>
T_3	$\rightarrow$	<code>next_available_pid = 2615</code>	$\leftarrow$
T_4	<code>return 2615</code>		<code>return 2615</code>
T_5	<code>child = 2615</code>		<code>child = 2615</code>

Problema: Lost Update

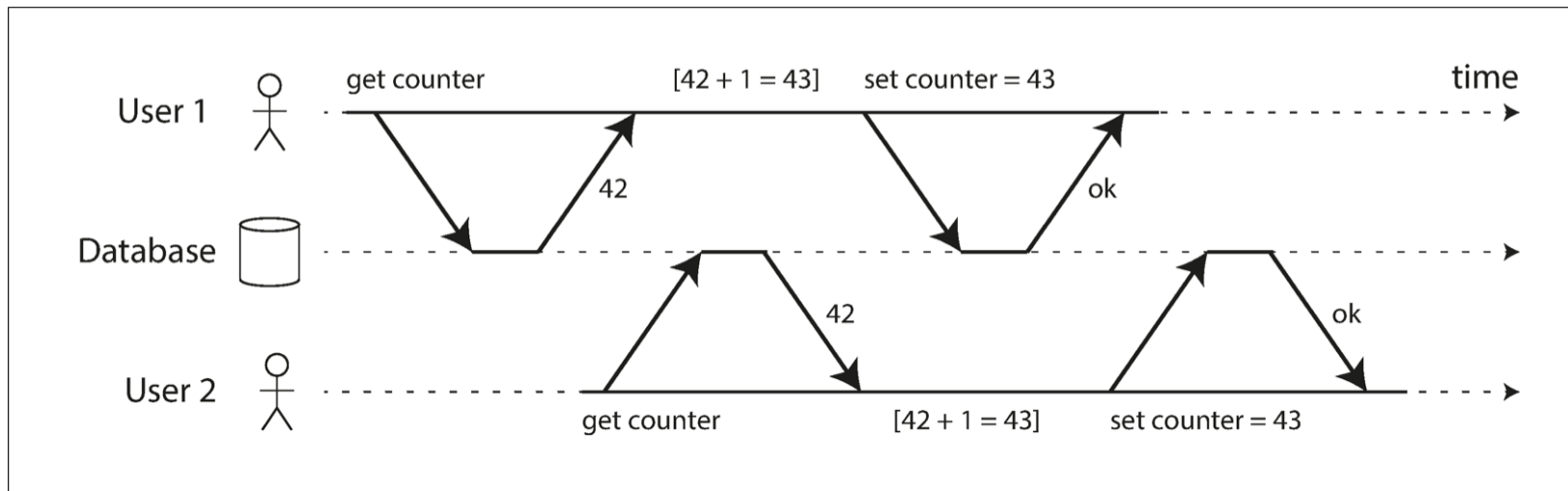


Figure 7-1. A race condition between two clients concurrently incrementing a counter.

Fonte da imagem: [Priyank Verma](#)

Problema: Lost Update - Exemplo

```
#include <stdio.h>
#include <pthread.h>

int contador = 0;

void* incrementa_contador(void* thread_id) {
    int tid = (int)(long)thread_id; // Cast para obter o ID da thread
    for (int i = 0; i < 10000; i++) {
        contador++; // Acesso direto à variável global sem proteção
    }
    printf("Thread %d: Contador final = %d\n", tid, contador);
    pthread_exit(0);
}

int main() {
    pthread_t thread1, thread2;
    pthread_create(&thread1, NULL, incrementa_contador, (void*)1); // Cria thread1
    pthread_create(&thread2, NULL, incrementa_contador, (void*)2); // Cria thread2
    pthread_join(thread1, NULL); // Espera a thread1 terminar
    pthread_join(thread2, NULL); // Espera a thread2 terminar

    printf("Valor final do contador: %d\n", contador); // O valor final será imprevisível
    return 0;
}
```

Seção Crítica

Bloco de código que acessa um recurso compartilhado.

```
entrar na região crítica  
    acessar ou modificar recurso compartilhado  
sair da região crítica
```

Exemplos de recursos compartilhados:

- Arquivos
- Bancos de dados
- Impressoras
- Variáveis compartilhadas
- Serviços de rede

Exclusão Mútua

Exclusão mútua significa garantir que apenas um processo por vez acesse uma seção crítica.

Objetivos:

- Evitar inconsistência de dados.
- Evitar conflitos no acesso a recursos.
- Garantir previsibilidade na execução.

Se Processo A está na seção crítica,
Processo B deve esperar.

Exclusão Mútua: Fluxo

1. Processo quer entrar: **pede permissão**
2. Sistema decide: **concede ou nega**
3. Processo executa: **acessa o recurso**
4. Processo termina: **libera o recurso**

Propriedades desejadas da exclusão mútua

Todo algoritmo de exclusão mútua deve satisfazer **3 propriedades**:

#	Propriedade	Descrição
1	Exclusão Mútua	Apenas 1 processo na seção crítica por vez. Sempre.
2	Progresso	Se nenhum está na seção crítica, quem quer entrar deve conseguir.
3	Espera Limitada	Nenhum processo espera para sempre. Starvation proibido.

- **Propriedades desejáveis:**

- **Tolerância a falhas:** se um nó cai, o sistema continua
- **Desempenho:** poucas mensagens trocadas

- **O que evitar:**

- **Deadlock:** dois processos esperando um pelo outro infinitamente
- **Starvation:** um processo nunca consegue entrar na seção crítica

Concorrência em Sistemas Distribuídos

Processos rodam em **servidores diferentes**, ligados por rede.

- **Sem memória compartilhada**: tudo passa por mensagens
- **Não há relógio global**: difícil ordenar eventos com precisão
- Mensagens podem chegar **fora de ordem ou se perder**
- Máquinas podem falhar de forma independente.

Precisamos de um **protocolo explícito** de coordenação.

Exemplo: Servidor A e Servidor B tentam atualizar o mesmo registro no banco de dados ao mesmo tempo. Qual atualização vale? Quem chegou primeiro?

Como organizar a ordem de eventos em um sistema distribuído?

Relógios de Lamport

Cada processo mantém um [relógios lógicos](#) usado em algoritmos de sincronização de relógio baseados na relação *happens-before* definida por [Leslie Lamport](#).

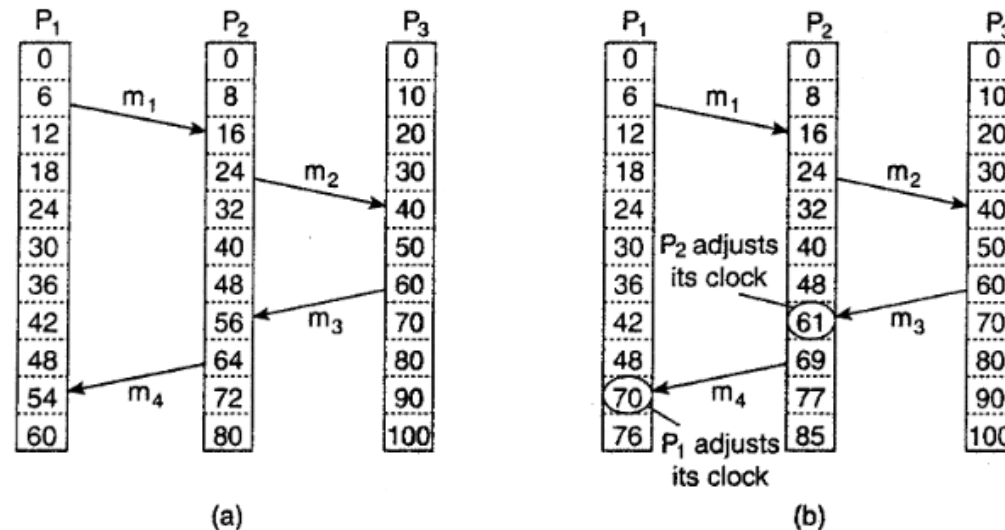


Figure 6-9. (a) Three processes, each with its own clock. The clocks run at different rates. (b) Lamport's algorithm corrects the clocks.

Na imagem: (a) Relógios de Lamport são sequências de números ou contadores. (b) O Algoritmo de Lamport corrige os relógios. Fonte: [StackOverflow](#).

Relógios de Lamport

Quando um processo envia uma mensagem, ele inclui o valor do seu contador (relógio) na mensagem.

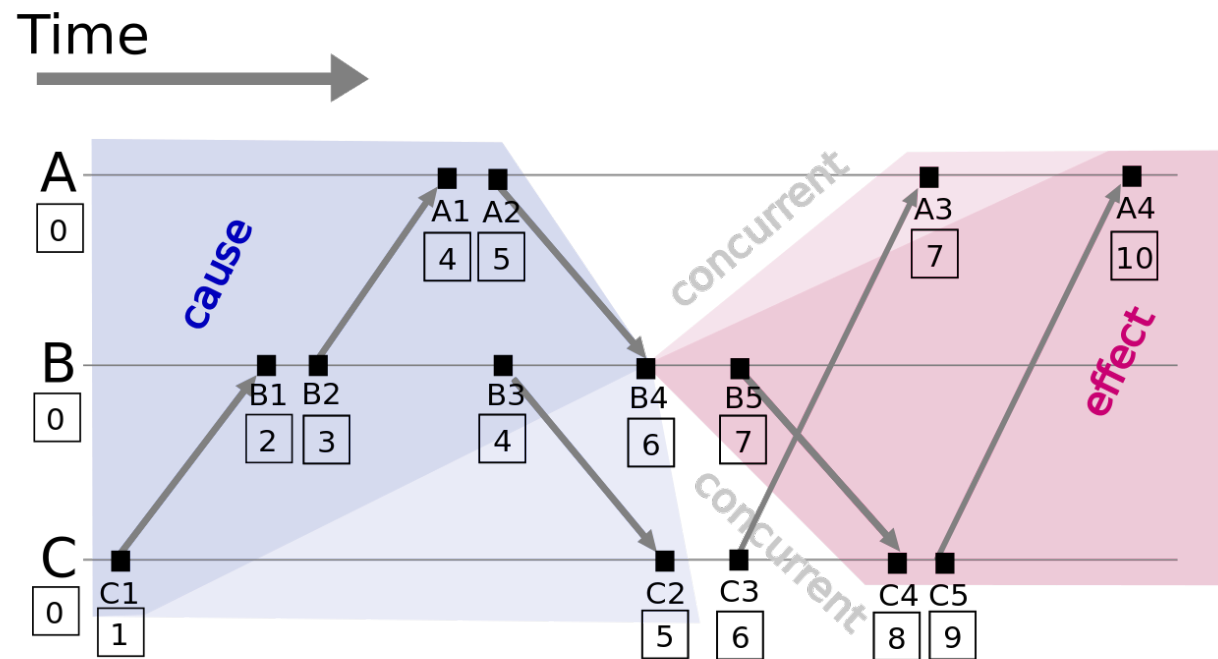


Diagrama de relógios de Lamport. Fonte da imagem: [Wikipedia](#)

Concorrência em Sistemas Distribuídos

Desafio principal: Em um sistema distribuído, como decidir quem pode acessar um recurso compartilhado?

Exemplo:

Nó A quer acessar um arquivo distribuído.
Nó B quer acessar o mesmo arquivo.
Nó C também quer acessar o mesmo arquivo.

Sem um controle adequado, os três podem tentar modificar o recurso **ao mesmo tempo**.

Exclusão Mútua em Sistemas Distribuídos

A exclusão mútua distribuída busca coordenar processos em máquinas diferentes. O problema é garantir que:

Apenas um processo distribuído acesse o recurso crítico por vez.

Isso deve ocorrer mesmo sem:

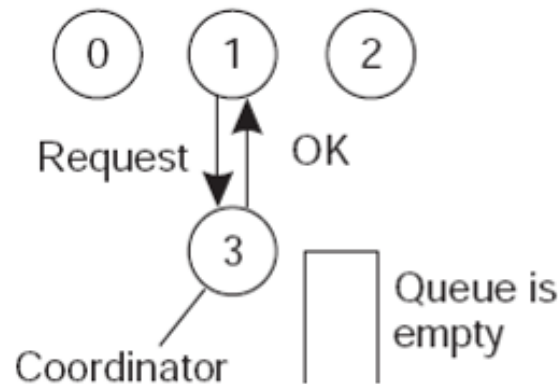
- Memória compartilhada.
- Relógio global confiável.
- Coordenador único obrigatório.

Abordagens para Exclusão Mútua Distribuída

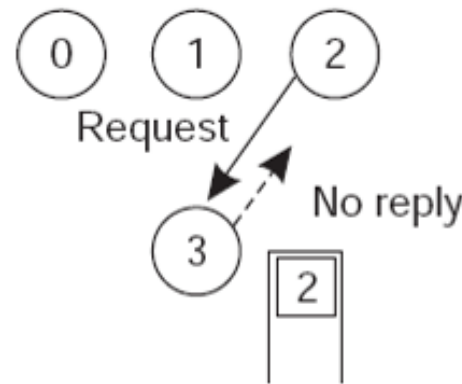
- **Algoritmo Centralizado:** Um nó especial gerencia todos os locks
 - **Vantagem:** Simples de implementar
 - **Desvantagem:** Ponto único de falha
- **Baseado em Permissão:** Processo pede permissão para **todos** os outros
 - **Vantagem:** Tolerante a falhas, totalmente descentralizado
 - **Desvantagem:** custoso, $2(N-1)$ mensagens por entrada
- **Baseado em Token:** Um token circula entre os processos
 - Quem tem o token pode entrar na seção crítica
 - **Vantagem:** Baixo overhead em pouca contenda
 - **Desvantagem:** Token pode ser perdido

Algoritmo Centralizado

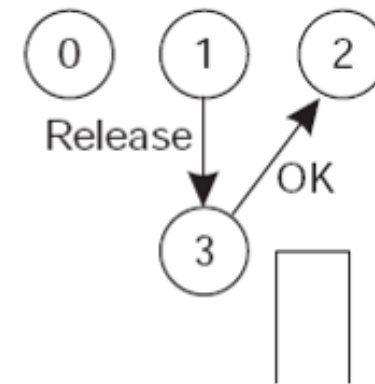
- Um nó especial é eleito **coordenador** (ou servidor de lock)
- Todos os outros nós são **clientes**: nunca se comunicam entre si
- O coordenador mantém uma **fila de espera** e controla quem pode entrar



(a)



(b)



(c)

Fonte da Imagem: quest10.com ↗.

Algoritmo Descentralizado

Um algoritmo descentralizado não depende de um único coordenador.

Ideia geral:

1. Um processo que deseja entrar na região crítica envia pedidos aos demais.
2. Os outros processos respondem autorizando ou negando temporariamente.
3. O processo entra na região crítica quando recebe permissões suficientes.
4. Ao sair, libera o recurso e avisa os demais.

Algoritmo Descentralizado: pedidos e permissões

Processo P1 quer entrar na região crítica.

P1 → P2: pedido de acesso

P1 → P3: pedido de acesso

P1 → P4: pedido de acesso

P2 → P1: permitido

P3 → P1: permitido

P4 → P1: permitido

P1 entra na região crítica.

A entrada só ocorre após receber as autorizações necessárias.

Algoritmo de Ricart-Agrawala (1981)

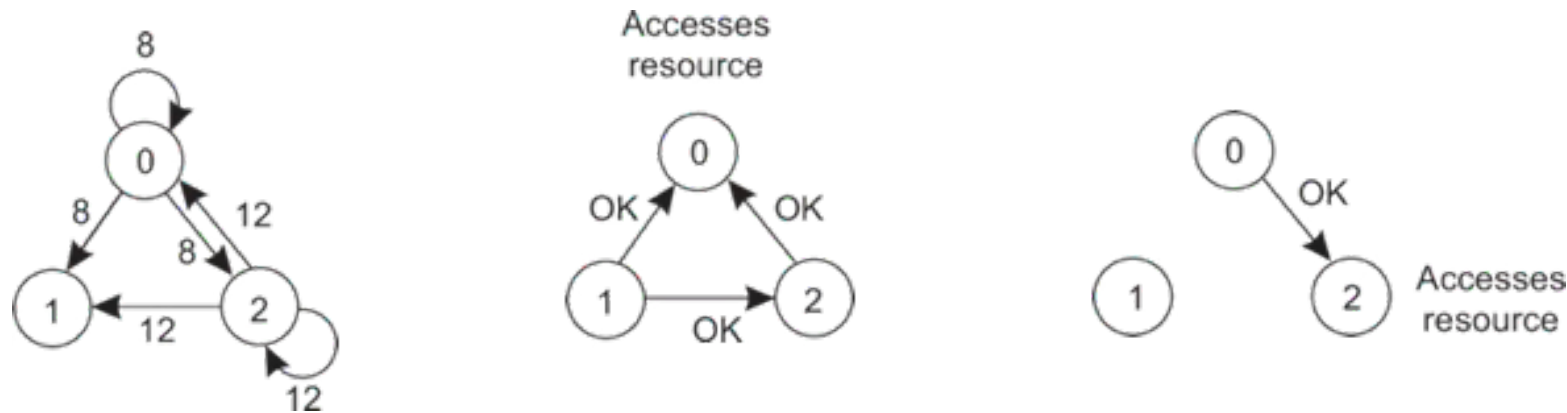
- Cada processo pede **permissão a todos os outros** antes de entrar
- Entra somente se receber **OK de todos**
- Usa **timestamps de Lamport** para desempatar pedidos simultâneos
- Processo com **menor timestamp** tem prioridade

Passo a passo:

1. P envia **REQUEST(ts, P)** para todos os nós
2. Cada nó responde **OK** — ou guarda na fila se tiver prioridade
3. P coleta OKs de **todos** → entra na seção crítica
4. Ao sair, P envia **OK** para os processos na fila

Regra de desempate: menor timestamp ganha. Em caso de empate de timestamp, menor ID de processo vence.

Algoritmo de Ricart-Agrawala: Exemplo



- Na esquerda: Há uma disputa por acesso a um recurso compartilhado entre dois processos.
- P0 possui o timestamp mais baixo, sendo, portanto, priorizado (ou selecionado).
- Ao finalizar sua operação, o processo P0 envia também uma confirmação de sucesso (OK), permitindo que o processo P2 prossiga com seu acesso.

Fonte da imagem: [Distributed Systems, Maarten van Steen and Andrew S. Tanenbaum](#)

Ricart-Agrawala: Pseudocódigo

Ao querer entrar na seção crítica:

ts = meu_timestamp + 1

enviar REQUEST(ts, eu) para TODOS

aguardar OK de TODOS os outros nós

Ao receber REQUEST(ts_deles, deles):

SE estou na seção crítica:

→ adicionar à fila (não responder agora)

SENÃO SE quero entrar E tenho prioridade:

→ (meu_ts < ts_deles)

OU (empate: meu_id < id_deles)

→ adicionar à fila (não responder agora)

SENÃO:

→ responder OK imediatamente

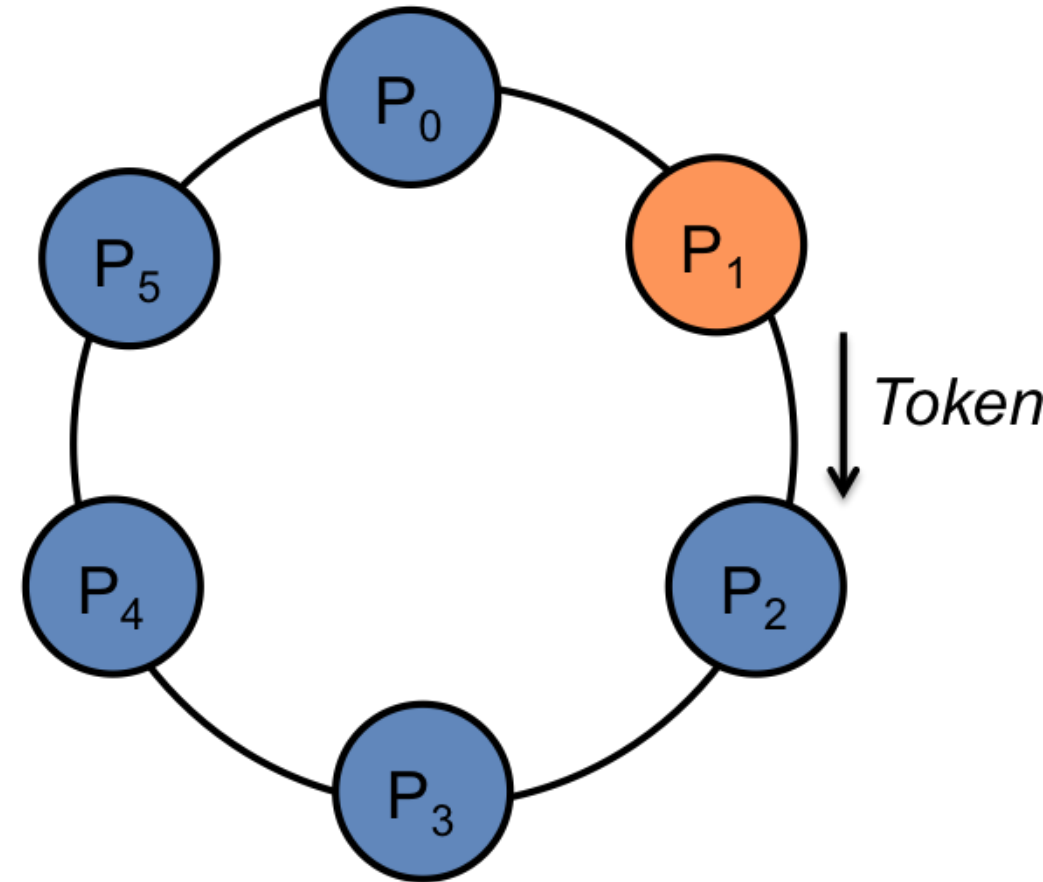
Ao sair da seção crítica:

enviar OK para todos os processos na fila

Algoritmo de Token em Anel

- Os processos formam um **anel lógico**
- Existe exatamente **1 token** no sistema
- O token passa de processo em processo **na ordem do anel**

Na imagem: O token circula em uma ordem definida.
Imagine uma linha de montagem onde o "sinal verde" (token) só é passado quando a peça está pronta.
Fonte da imagem: [Pinclipart](#) .



Algoritmo de Token em Anel

O token funciona como uma “chave” que autoriza o uso do recurso compartilhado.

```
funcao Processo_i(Recebeu_Token):  
    // 1. Entra na Seção Crítica  
    Executar_Tarefa();  
    // 2. Passa o Token adiante  
    Enviar_Token(Processo_(i+1));
```

Funcionamento:

1. Os processos são organizados em um círculo (anel).
2. Apenas quem possui o *Token* pode entrar na Seção Crítica.
3. Ao terminar, o processo deve passar o *Token* para o próximo da fila.
4. Se não precisa acessar o recurso, apenas repassa o *Token*.

Algoritmo de Token: Vantagens e Desvantagens

Vantagens:

- Garante exclusão mútua de forma simples.
- Evita conflito direto entre múltiplos pedidos.
- Pode reduzir o número de mensagens em alguns cenários.

Ideia importante: não há disputa simultânea se só existe um token válido.

Desvantagens:

- O token pode ser perdido por falha de um processo.
- Um processo pode demorar a repassar o token.
- É necessário detectar e regenerar o token com cuidado.
- Se houver dois tokens por erro, a exclusão mútua pode ser violada.

Algoritmo Descentralizado

Evita dependência do **controlador**

- **Princípio:** Assuma-se que cada recurso é replicado N vezes.
 - Cada réplica possui seu próprio coordenador (ou *coordinator*).
 - Portanto, o acesso ao recurso exige um voto majoritário (m) dos coordenadores, onde $m > N/2$.
 - Um coordenador deve sempre responder imediatamente a qualquer requisição.
 - Exemplo: Se você tem 5 cópias de um dado ($N = 5$), você precisa da maioria (3 votos) para confirmar se o acesso foi bem-sucedido.
- **Suposição sobre Falhas e Estado:** Quando um coordenador falha, ele se recuperará rapidamente, mas terá perdido o registro das permissões ou do estado que havia concedido antes da falha.

Algoritmo Descentralizado (cont.)

Assuma N processos ($P_1, P_2, \dots, P_N$) e todos querem entrar na Seção Crítica (SC).

Fase 1: O Processo Solicita Acesso (Request)

- Um processo (P_i) decide que precisa acessar o recurso. Ele não pode simplesmente entrar; ele deve pedir permissão.
 - **Geração de Timestamp:** P_i gera um *timestamp* único e crescente (ex: **Timestamp = [Hora Atual, ID do Processo]**). Isso garante uma ordem determinística para os pedidos.
 - **Envio da Requisição:** P_i envia mensagens de "Solicitação" (Request) a todos os outros processos (P_j).
- **Exemplo:** P_1 quer entrar na SC às 10:05:30. Ele envia um pedido para $P_2, P_3, \dots$

Algoritmo Descentralizado (cont.)

Fase 2: Os Processos Avaliam a Solicitação (Voting)

- Os processos receptores (P_j) recebem o pedido e precisam decidir se podem conceder acesso ou não.
 - **Verificação de Conflito:** P_j verifica seu próprio estado. Ele está ocupado? Se estiver, ele deve responder "Não" (ou adiar).
 - **Regra da Ordem:** Se P_j também tiver feito um pedido recentemente, ele compara os *timestamps*. O processo com o *timestamp* mais antigo tem prioridade.
 - **Resposta de Concessão:** Se P_j estiver livre e não houver conflito de ordem, ele envia uma mensagem de "Permissão" (Grant) para P_i .
- **Exemplo:** P_2 recebe o pedido de P_1 . P_2 verifica: "Eu estou livre? Sim. P_1 's timestamp é mais antigo que qualquer coisa que eu tenha feito? Sim." $\rightarrow$ P_2 envia "Permissão" para P_1 .

Algoritmo Descentralizado (cont.)

Fase 3: Acesso e Conclusão (Access & Release)

- O processo original (P_i) só pode entrar na SC quando coletar o número suficiente de permissões.
 - **Coleta de Votos:** P_i espera receber um "Grant" de uma maioria dos processos (ou, em modelos mais simples, de todos os processos).
 - **Entrada na SC:** Ao coletar todas as permissões necessárias, P_i entra na Seção Crítica e executa o código.
 - **Liberação:** Assim que P_i termina, ele envia mensagens de "Liberação" (Release) para todos os processos vizinhos ou interessados, sinalizando que o recurso está livre.
- **Exemplo:** P_1 recebe 4/5 permissões. Ele entra na SC e processa a transação bancária. Ao sair, ele envia um "Liberado" geral.

Algoritmo Descentralizado

- Toda requisição (**Request**) para acessar a região crítica deve ser respondida (**Response**)
 - Usa relógio lógico de Lamport para ordenar as solicitações
- Se processo obtiver a maioria M das permissões então processo entra na região crítica
- Senão, se outro processo obteve a maioria, então processo espera pela sua vez para entrar na região crítica.
- Se nenhum processo obteve a maioria das permissões então processos devem reordenar as filas de requisição (**Yield**)

$$Yield = Release + Request$$

Algoritmo Descentralizado

- **Vantagem:** maior resistência à queda de um coordenador
- **Desvantagem:** possível “perda” de memória com a queda de múltiplos coordenadores

Critérios de Avaliação dos Algoritmos

Ao comparar algoritmos de exclusão mútua distribuída, analisamos:

- Número de mensagens trocadas.
- Tempo de espera para entrar na região crítica.
- Tolerância a falhas.
- Risco de deadlock.
- Justiça entre os processos.
- Escalabilidade com muitos nós.

Comparação entre Algoritmos

Algoritmo	Mensagens por entrada/saída	Espera antes que possa entrar (num. de mensagens)	Problemas
Centralizado	3	2	Queda do coordenador
Distribuído	$2(N - 1)$, onde N é o número de nós	$2(N - 1)$	Queda de qualquer processo
Token	1 a ∞	0 a $N - 1$	Perda do token e queda de qualquer processo
Descentralizado	$2kN + (k - 1)N/2 + N$, onde K é o num. de tentativas	$2kN + (k - 1)N/2$	Inanição, baixa eficiência

Exercício 1 (discussão em sala)

Considere três processos distribuídos: P1, P2 e P3.

Todos desejam acessar o mesmo arquivo compartilhado.

Perguntas:

1. O que pode dar errado se não houver exclusão mútua?
2. Qual parte da operação representa a região crítica?
3. Como um algoritmo baseado em token resolveria esse problema?

Exercício 2 (discussão em sala)

Analise a situação:

P1 está com o token.
P2 precisa acessar a região crítica.
P3 também precisa acessar a região crítica.
P1 falha antes de repassar o token.

Perguntas:

1. Qual é o problema gerado?
2. Por que não é seguro simplesmente criar um novo token imediatamente?
3. Que tipo de mecanismo adicional seria necessário?

Conclusão

- **Race condition:** dois processos acessam o mesmo dado → resultado incorreto
- **Seção crítica:** trecho de código que acessa recurso compartilhado
- **Exclusão mútua:** garantir que apenas 1 processo acessa por vez
- **3 requisitos:** exclusão mútua, progresso, espera limitada
- **Algoritmo de Ricart-Agrawala:** pede permissão a todos, timestamps para desempatar
- **Algoritmo de Token em Anel:** token circula, quem tem o token pode entrar

Pergunta: O que acontece se o nó que tem o token falhar *durante* a seção crítica? Como o sistema detecta essa falha e regenera o token com segurança?

Material Adicional

- Livro Coulouris: Capítulo 15, seção 15.2
- Livro Tanenbaum: Capítulo 6, seção 6.3
- Shi-Ding Lin, Qiao Lian, Ming Chen, Zheng Zhang. A Practical Distributed Mutual Exclusion Protocol in Dynamic Peer-to-Peer Systems. Proceedings 3rd International Workshop on Peer-to-Peer Systems (IPTPS'04), pp. 11-21, 2004.
- [Exclusão mútua em sistemas distribuídos - Prof. Dorgival Guedes \(UFMG\)](#) 

Dúvidas e Discussão

Questões para Estudo

Questão 1

Em um sistema distribuído real, o que é mais difícil de lidar?

- Muitos processos concorrendo?
- Falhas de comunicação?
- Perda do token?
- Falta de relógio global?

Questão 2

Explique a principal diferença arquitetural e de resiliência entre usar um **Lock Manager Centralizado** versus um sistema baseado em **Passagem de Token**. Em qual cenário você preferiria cada abordagem?

Descreva o passo a passo (em 4 etapas) que ocorre desde o momento em que o Processo A recebe o Token até o momento em que ele passa o Token para o Processo B, garantindo que nenhum outro processo possa interferir nesse intervalo.

Questão 3

Considere um sistema distribuído usando Passagem de Token. Se o **Processo C** falhar *depois* de ter executado a Seção Crítica, mas *antes* de passar o Token para o Processo D, qual é o estado do sistema? Como esse problema pode ser mitigado em teoria (sem detalhar algoritmos complexos)?

Questão 4

Em um ambiente onde os processos são muito numerosos e as operações na Seção Crítica são extremamente rápidas (milissegundos), qual dos mecanismos de exclusão mútua (Centralizado ou Token) tende a gerar mais *overhead* (custo extra de comunicação)? Justifique sua resposta.